

Data Engineer · 신입 지원

데이터엔지니어 포트폴리오

수집 · 정제 · 모델링 · 검색 · 배포

공공 API에서 실서비스까지, 데이터 파이프라인 전 과정을
직접 설계하고 운영한 준비된 데이터 엔지니어

전예원

수원대학교 컴퓨터소프트웨어학과 (2027.02 졸업예정) · DAsP
2026 한이음 드림업 LawSight 팀장 · 중간평가 상위 30% 우수 프로젝트

서비스
UI · API · 배포 · 운영

기능
검색 · RAG · 분석 로직

데이터
수집 · 정제 · DB 설계 · 벡터 인덱스

▲ 전체 프로젝트 데이터 파트 주도적 수행

목차

00 **핵심 역량 요약**
학력 · 자격 · 기술 스택 · 프로젝트별 수행 역할

01 **LawSight — AI 판례 분석 시스템 (팀장)**
상황 → 해결 과제 → 액션 (수집 · 전처리 파이프라인 · 저장소 구성 · 장애 대응) → 결과

02 **My Task Pilot — AI Agent 업무 자동화 (팀원)**
상황 → 해결 과제 → 액션 (비정형 데이터 파이프라인 · AWS EC2 · 정합성 검증) → 결과

03 **영화 예매 플랫폼 — 표준 기반 데이터 모델링 (개인)**
상황 → 해결 과제 → 액션 (논리 · 물리 ERD · DB 구축 · 이슈 해결) → 결과

핵심 역량 요약

데이터 수집 · 정제 · 모델링 · 검색 인덱스 · 배포를 Full Cycle 로 구축 / 운영

학력
수원대학교 컴퓨터소프트웨어학과
2023.03 - 2027.02 졸업예정 (국어국문학과 입학 후 2학년 전과)

자격 · 교육
DAsP 취득 · SQLD/OCP 2026.11 응시 준비
오라클 DB 전문가 과정 수강 중 (아이티윌, 5개월, 26.6~26.11)
[오라클 교육 학습 일지](#)

기술 스택
DB PostgreSQL(+pgvector) · Oracle · ERD 설계 (DA# Modeler)
Data ChromaDB · KoSBERT · 벡터 검색 · Pandas
Backend/AI Python · FastAPI · Spring Boot/MyBatis · ML
Infra AWS EC2 · Linux · Git/GitHub · Notion

활동
2026 한이음 드림업 2개 프로젝트 수행 (진행중, 26.4~26.10)
- LawSight 팀장 역할, MyTaskPilot 팀원 역할
* 2개 과제 중간평가 상위 30% 우수 프로젝트 및 장려상 후보 선정

	프로젝트	구분 · 기간	역할	핵심 성과
실전 단계	LawSight AI 판례 분석 · 소송전략	한이음 드림업 · 4인 2026.04 ~ 10 (7개월)	팀장 (PM) Data · DB · Infra 총괄	판례 41,438건 파이프라인 검색 응답 0.68초 · 실서비스 배포 https://www.lawsight.co.kr
	My Task Pilot AI Agent 업무 자동화	한이음 + 졸업작품 · 3인 2026.04 ~ 10 (7개월)	팀원 (Data / AI) DB 설계 · RAG 검증	PostgreSQL+pgvector 6T 운영 DB 전수 정합성 검증 https://www.mytaskpilot.kr
응용 단계	영화 예매 플랫폼 데이터 설계 역량 강화	개인 · 3학년 겨울방학 2026.01 ~ 02 (2개월)	단독 수행	논리 · 물리 ERD 설계 PostgreSQL 구축 https://movie-booking-platform-ui.vercel.app
	맞춤형 식단 관리 웹 공공데이터 연동	개인 · 3학년 여름방학 2025.07 ~ 08 (2개월)	단독 수행	공공데이터 1,000건 연동 집계 쿼리 · 시각화
	신용 리스크 예측 알고리즘분석 과제	개인 · 3학년 수강과목 2025.10 ~ 11 (2개월)	단독 수행	15만건 · SMOTE 보정 Test AUROC 0.8585
기초 단계	CS & Language	개인 · 2학년	단독 수행	운영체제, 리눅스, 네트워크, 자료구조 C, Java, Python

프로젝트별 수행 역할

서비스 구성 및 프로젝트 단계별 수행 역할 : 데이터 파트 주도적 역할 수행

프로젝트	서비스 구성별 수행 범위			프로젝트 단계별 수행 범위		
	데이터	기능	서비스	분석	설계	개발
LawSight	주도 수집 · 정제 · DB · 벡터 DB	협업 검색 연동 · 병목 개선	주도 EC2 배포 · 운영	주도 요구사항 35 건 총괄	협업 테이블 / 기능 / 화면 설계	협업 파이프라인 / 기능 / 화면
My Task Pilot	주도 수집 · pgvector · 전환	협업 RAG 개인화 기획 · 검증	협업 제출 문서 총괄	협업 요구사항정의	협업 테이블 / 기능 / 화면 설계	협업 기능 구현 / 테스트
영화 예매 플랫폼	주도 논리 · 물리 ERD	주도 예약 · 좌석 JOIN 쿼리	주도 웹 화면 구현	주도 요구사항 정의서	주도 DA# Modeler/ 화면 설계	주도 Node.js · PostgreSQL
맞춤형 식단 관리 웹	주도 공공데이터 수집	주도 칼로리 · 영양 집계	주도 차트 시각화	주도 요구사항 정의서	주도 기능 / 화면 설계	주도 Spring Boot·MyBatis
신용 리스크 예측	주도 결측 · 불균형 보정	주도 10 개 모델 비교 · 선정	해당없음	주도 데이터 탐색 · 목표 설정	주도 검증 전략 설계	주도 모델링 · Test/ 검증
全 프로젝트 데이터 파트 주도						

01

LawSight — AI 판례 분석 및 소송 전략 시스템

2026 한이음 드림업 · Team 404 (4 인) · 팀장(PM) 겸 데이터 · DB · 배포 인프라 총괄 · 2026.04 - 진행 중

LawSight > AI 판례 분석 및 소송전략 시스템

변호사가 자연어로 사건을 입력하면 유사 판례를 찾고 IRAC 형식의 소송 전략 리포트를 생성하는 AI 리서치 보조 서비스

구분	본인 역할	대상 사용자	기술 스택
2026 한이음 드림업 (과기정통부) · Team 404 · 4인 · 2026.04~10	팀장 (PM) 겸 Data · DB · 배포 인프라 총괄	개인 · 소형 로펌 변호사	PostgreSQL · ChromaDB · KoSBERT · BM25+RRF · FastAPI · GPT-4o · AWS EC2

상황 판례 리서치는 수작업, 원천 데이터는 비정형

- 변호사의 판례 검색 · 비교는 사건마다 수작업
현직 변호사 인터뷰 결과 “사실관계가 비슷한 판례”와 “근거 없는 답변 (환각) 배제”가 핵심 니즈
- 원천은 국가법령정보 공공 API — 목록과 본문이 분리
판례 목록 조회와 본문 조회가 별도 API, 판결문은 장문의 비정형 텍스트
- 표기 불일치 · 중복이 섞인 원시 데이터
같은 법원이 여러 이름으로 기록 (원시 104 종), 동일 판례 중복 수집 발생
- 제한된 자원과 일정
4인 · 7개월, 클라우드 지원금 범위의 EC2 단일 인스턴스에서 서비스 운영

해결과제 데이터가 검색 품질을 결정한다

- 1 대규모 판례 수집 자동화
2 단계 API 호출로 수만 건 판례를 중단 없이 수집 · 재실행 가능한 구조
- 2 비정형 판결문 정제 · 표준화
파싱 · 법원명 표준화 · 중복 제거 · 테이블 간 정합성 보장
- 3 정형 + 벡터 저장소 설계
메타 · 본문은 PostgreSQL, 의미 검색은 ChromaDB 로 역할 분리
- 4 제한 자원에서 실서비스 운영
검색 응답 속도 · 메모리 안정성 확보, HTTPS 배포 · 팀 산출물 관리

NEXT 액션 ① 수집 · 전처리 파이프라인 > 액션 ② 저장소 · 서비스 구성 > 액션 ③ 운영 장애 대응 > 결과

LawSight > 수집 · 전처리 파이프라인 설계 및 개발

데이터 수집처리 정의서로 기준을 먼저 확정한 뒤, 수집 → 파싱 → 정제 · 검증 → 적재 → 임베딩 5 단계를 Python 으로 직접 구현



파이프라인 설계 원칙

문서 우선 설계

데이터 요구사항 (FR-DATA) 7 건과 데이터 수집처리 정의서 (대상 · 방법 · 전처리 · 정합성 기준) 를 먼저 확정하고 코드로 구현

재실행 가능한 파이프라인

단계별 스크립트 분리 + 체크포인트 재개 구조 → 72 만 청크 전면 재생성도 처음부터 다시 돌리지 않고 안전하게 수행

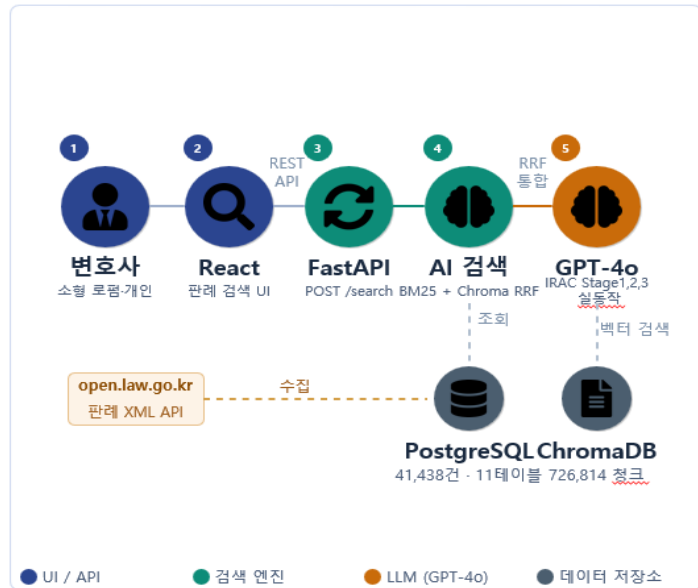
검증을 파이프라인 안에

적재 후 정합성 검증 스크립트를 실행해 테이블 간 1:1 관계 · 중복 · 누락을 매 수집 주기마다 확인

LawSight > 저장소 · 서비스 구성

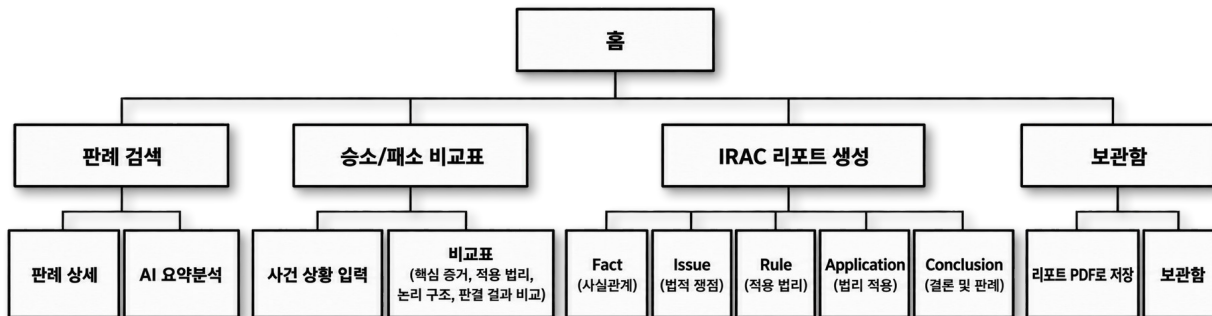
데이터 계층 (수집 ~ 벡터 인덱스) 을 단독 설계·구축하고, 기능 계층은 성능 개선, 서비스 계층은 배포 인프라를 담당

서비스 구성도

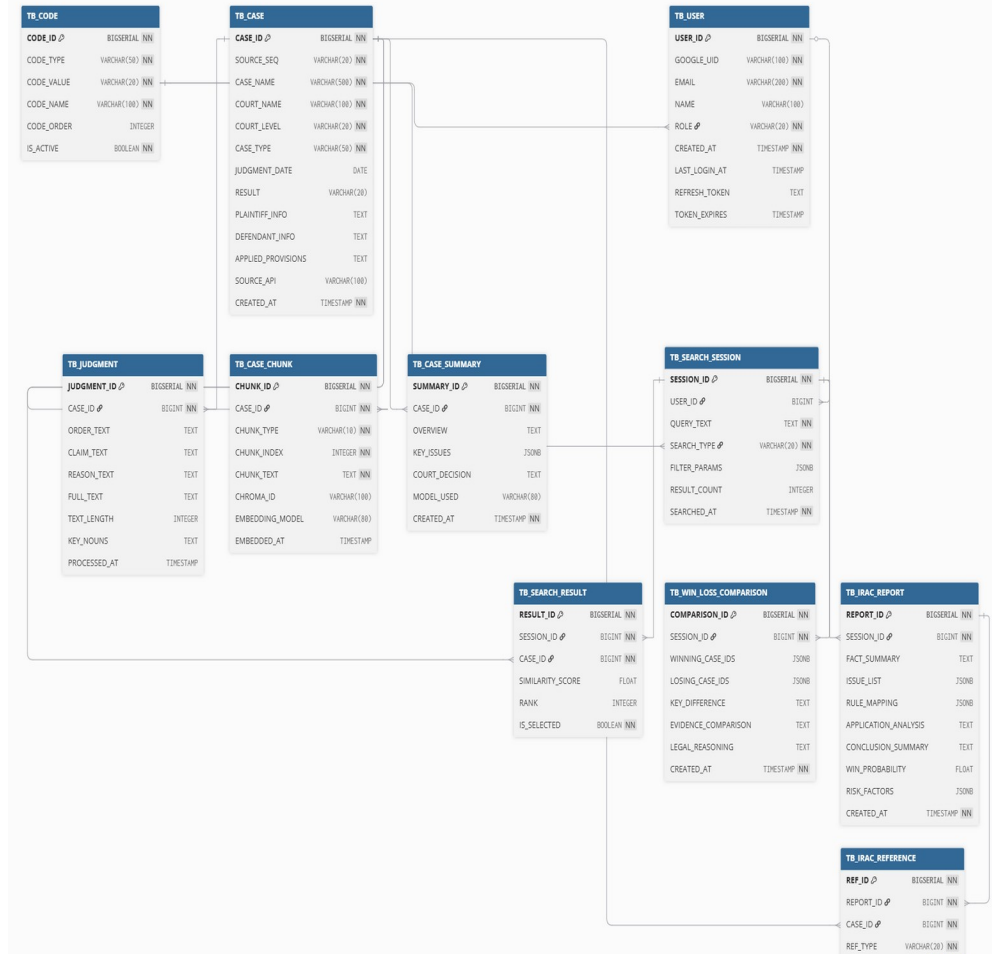


- 판례 검색 요청**
변호사가 자연어로 사건 내용 입력
- API 전달**
React → FastAPI POST /search
쿼리 임베딩 생성 후 검색 엔진 호출
- 하이브리드 검색**
BM25 키워드 + ChromaDB 벡터 병렬
RRF 통합 → Top-K 판례 반환
- 판례 조회 · 반환**
PostgreSQL 원문 조회
ChromaDB 벡터 유사도 점수 포함
- IRAC 분석 리포트**
GPT-4o → 쟁점(Issue) 추출
IRAC Stage1,2,3 실행작

메뉴 구성도



테이블 구성도 (ERD)



LawSight > 운영 장애 대응

문제 → 원인 → 조치 → 효과를 측정 가능한 지표로 정리

Case 1 요청별 BM25 인덱스 재로딩 ▮ 병목 원인 분석 · 개선

문제 검색 요청마다 383MB 인덱스를 디스크에서 재로딩 → 20~60 초 타임아웃

원인 인덱스 로딩이 요청 처리 경로 안에 있어 매 요청이 I/O + 역직렬화 비용을 전부 부담

조치 검색 서비스를 모듈 레벨 인스턴스로 전환, 서버 시작 시 1회만 로딩

효과

0.68 초

20~60 초 → 0.68 초

Case 2 임베딩 모델 오혼입 ▮ 726,814 개 청크 전면 재생성

문제 최종 점검에서 7,501 청크가 설계 (KoSBERT) 와 다른 MiniLM 으로 임베딩된 것을 발견

원인 기존 실행 환경의 .env 기본값이 MiniLM 을 가리킨 상태로 임베딩이 수행된 것을 확인

조치 .env 기본값 원인 확인 → 모델 코드 고정, 체크포인트를 실제 저장 건수 기준으로 단일화

효과

726,814

청크 전량 재생성

Case 3 메모리 부족 (OOM) → 제한 자원 환경에서의 팀 공동 대응

문제 RAM 4GB 에서 인덱스 · 모델 · 벡터 DB 동시 상주 → OOM killer 가 백엔드 종료

원인 인스턴스 증설 후에도 재발해 단순 사양 문제만은 아님을 확인. 메모리 상주 객체와 전체 검색 범위를 함께 점검

조치 journalctl 추적, t3.medium 증설 · swap 4GB 구성 (팀 : BM25 인덱싱 범위 축소)

효과

687MB

swap 사용 3.2GB → 687MB

Case 4 ChromaDB 대량 적재 세그폴트 ▮ 분할 적재 및 Rebuild 수행

문제 단일 컬렉션 적재 중 세그폴트 반복, EC2 기본 Python 3.14 의존성 설치 실패

원인 DB 무결성은 정상이고 Python 3.11/3.14 에서 모두 재현되어 단일 컬렉션 규모 · 메모리 한계를 우선 원인으로 판단 / 3.14 는 일부 Rust 기반 의존성의 사전 빌드 미지원

조치 2 개 컬렉션 (571,603 / 152,326) 분할 적재, Python 3.11 소스 빌드 + 전용 venv

효과

723,929

청크 적재 완료

판례 41,438 건 기반 하이브리드 검색 서비스를 실서비스로 배포하고, 한이음 중간평가 상위 30% 선정

판례 수집 (초기 3,503 → 최종)

KoSBERT 임베딩 체크

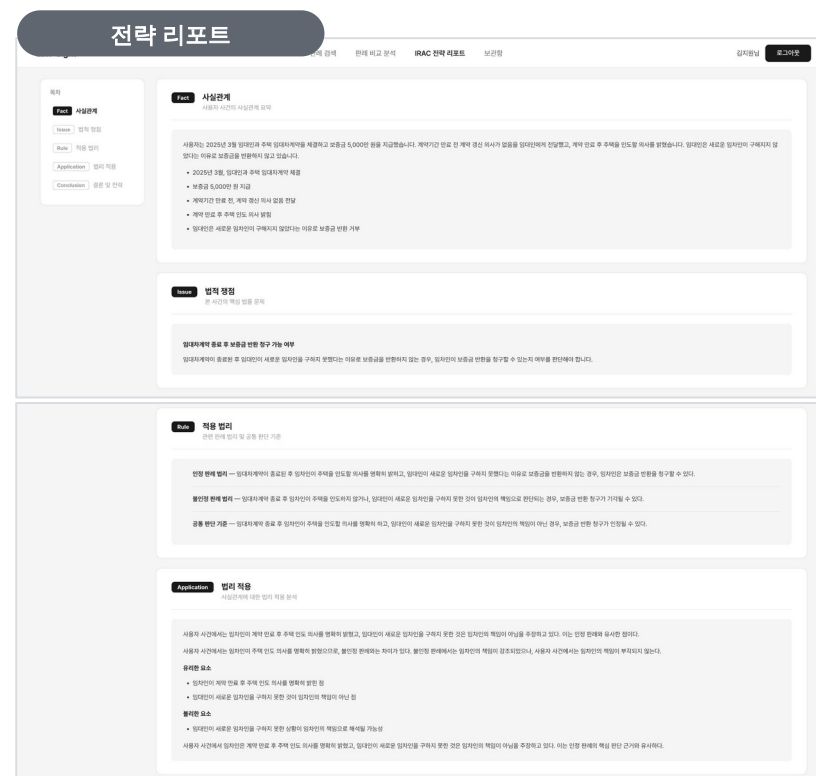
PostgreSQL 테이블 / 인덱스

법원명 원시 유형 표준화

검색 응답 (20~60 초 → warm)

한이음 중간평가 우수

- www.lawstight.co.kr 실서비스 배포 (EC2 t3.medium · nginx · HTTPS)
- 검색 타임아웃 해소 — BM25 1 회 로딩으로 응답 0.68 초 · 실패율 0%
- 개발설계서 · 완료보고서 제출, 중간평가 상위 30% · 장려상 후보
- “ 확인된 진척”만 기록하는 문서 · 코드 정합성 점검 프로세스를 팀에 정착
- 배운 점 : 검색 품질은 모델보다 데이터 정제 · 재실행 가능한 파이프라인에서 결정



LawSight > 프로젝트 수행 및 산출물

팀장으로서 전 단계 산출물을 총괄하고, 팀원별 산출물 검토 및 보완 수행

01 분석

본인 주도 4 · 팀원 1 / 전체 5

● 주도

요구사항 정의서 총괄

5 개 모듈 · 35 개 기능 요구사항 (FR) 정의

● 주도

데이터 요구사항 (FR-DATA)

수집 · 전처리 · 인덱싱 7 개 요구사항

● 주도

시장 / 기술 환경 분석

경쟁 서비스 3 사 기능 · 가격 비교

● 주도

현직 변호사 인터뷰

무환각 · 사실관계 유사성 니즈 도출

○ 팀원

유즈케이스 정의서

UC-01 ~ UC-12 액터 · 관계 정의

02 설계

본인 주도 4 · 협업 1 / 전체 6

● 주도

서비스 구성도 · 흐름도

수집 · 인덱싱 파이프라인 흐름 설계

● 주도

데이터 수집처리 정의서

수집 대상 · 방법 · 전처리 · 정합성 기준

● 주도

ERD v1 → v2 (11 테이블)

PostgreSQL + ChromaDB 저장소 분리 설계

● 주도

테이블 / 필드 / 도메인 정의서

데이터사전 포함 명세서 4 시트

① 협업

메뉴 구성도 · 화면 설계서

화면 ID ↔ 기능 ID 상호참조

○ 팀원

IRAC 알고리즘 명세서

LLM 분석 체인 설계

03 개발

본인 주도 4 · 협업 1 / 전체 6

● 주도

수집 · 전처리 프로그램

collect_cases.py · parse_judgment · 정규화

● 주도

DB 구축 (DDL · 인덱스)

11 테이블 · FK CASCADE · GIN 인덱스

● 주도

임베딩 · 벡터 인덱싱

KoSBERT 재생성 · ChromaDB 분할 적재

● 주도

AWS 배포 · 운영

EC2 · nginx · systemd · HTTPS

① 협업

검색 연동 · 성능 개선

서비스 인스턴스 재사용 · 0.68 초

○ 팀원

IRAC 체인 · React UI

LLM 분석 · 화면 구현

전 단계 공통 — 프로젝트 관리 (PM)

일정 · 이슈 관리

1 주 단위 회의 · Notion WBS/ 간트

확인된 진척만 반영

문서 · 코드 · 배포 상태 교차 검증 절차

중간 합류 온보딩

Notion 문서 · pg_dump 스키마 공유

성과

중간평가 상위 30% · 최종 산출물 제출

02

My Task Pilot — AI Agent 업무 자동화

2026 한이음 드림업 겜 졸업 프로젝트 · Team404 (3 인) · Data/AI 담당 — DB 설계 · 관리, RAG · 멀티모달 파이프라인 · 2026.04 ~ 진행중

MyTaskPilot > AI Agent 업무 자동화 서비스

텍스트 · 음성 (STT) · 이미지 (OCR) 입력을 분석해 일정 · 이메일 · 할 일을 자동 등록하는 개인 업무 비서 서비스

구분

2026 한이음 드림업 + 졸업 프로젝트 · Team404
· 3 인 · 2026.04~10

본인 역할

Data / AI — DB 설계 · 관리, RAG · 멀티모달
파이프라인, 문서 총괄

대상

반복 업무를 자연어로 처리하려는
개인 사용자

기술 스택

PostgreSQL + pgvector · Whisper · GPT-4o-mini
Vision · text-embedding-3-small · AWS EC2

상황

입력은 비정형, 저장소는 프로토타입 수준

■ 반복 업무가 여러 앱에 흩어져 수작업 처리

일정 등록 · 할 일 정리 · 이메일 초안을 사용자가 매번 직접 입력

■ 입력이 음성 · 이미지 · 자유 텍스트 — 비정형 데이터

그대로는 DB 에 저장 · 조회할 수 없어 텍스트화와 구조화가 먼저 필요

■ 초기 구현은 로컬 SQLite

다중 사용자 · 벡터 검색 · 상시 운영이 필요한 실서비스 요구를 감당하기 어려움

■ AI 오판 시 일정 · 메일이 잘못 실행될 위험

자동 실행 전에 사용자 승인 (HITL) 과 서버 측 권한 검증 필요

해결 과제

비정형 입력을 신뢰할 수 있는 데이터로

1

비정형 입력의 정형화

음성 · 이미지 · 텍스트를 일정 / 할 일 / 메일 레코드로 변환하는 파이프라인

2

운영형 데이터 모델로 전환

SQLite → PostgreSQL + pgvector, users 중심 6 테이블 설계

3

개인화 RAG

사용자 이력을 임베딩해 유사 이력 top-3 를 응답에 반영

4

운영 DB 신뢰성

클라우드 운영 DB 와 설계 문서 일치, 권한 우회 취약점 차단

NEXT 액션 ① 비정형 데이터 파이프라인 · AWS EC2 > 액션 ② 운영 DB 정합성 · 보안 검증 > 결과

MyTaskPilot > 비정형 데이터 파이프라인 · AWS EC2

음성 · 이미지 · 자유 텍스트를 정형 레코드와 벡터로 바꿔 저장하고, 상시 운영이 필요한 DB · 벡터 검색은 EC2 에 구성

비정형 데이터 파이프라인

입력 수집

텍스트 · 음성 · 이미지 — 형식이 제각각인 비정형 입력
채팅 텍스트, 음성 녹음, 일정 캡처 · 문서 사진을 하나의 입력 채널로 수용

텍스트화

Whisper STT · GPT-4o-mini Vision OCR
음성 · 이미지를 텍스트로 변환, 샘플 입력으로 STT / OCR 정확도 검증

구조화

Function Calling 의도 분류 → 필드 추출 → HITL 승인
일정 / 할 일 / 메일 의도와 제목 · 일시 등 필드를 JSON 으로 추출, 사용자 승인 후 확정

정형 저장

PostgreSQL 6 테이블 (users 중심)
schedules · todos · email_drafts 에 저장, agent_executions 에 실행 이력 기록

벡터화 RAG

text-embedding-3-small (1536 차원) → pgvector
user_embeddings 에 저장, 코사인 유사도 top-3 이력을 응답에 반영해 개인화

■ 본인 주도 (저장 · 벡터 스키마) ■ 기획 · 검증 참여 ■ 팀원 구현 · 본인 연동

226 건

합성 검증 데이터 · 페르소나 3 종

0.1~0.2 초

pgvector 유사도 검색 (테스트)

6 테이블

정형 레코드 · 실행 이력 · 임베딩

왜 클라우드 (AWS EC2) 인가

로컬 SQLite 프로토타입 → 상시 운영 클라우드 DB 로 전환

1

pgvector 확장을 직접 설치 · 제어

PostgreSQL 버전과 vector 확장을 직접 설치 · 관리할 수 있는 서버 환경이 필요

2

24 시간 접속되는 실서비스

mytaskpilot.kr 도메인 · Nginx · HTTPS, Google OAuth 콜백 등 상시 응답 서버 필요

3

팀 공용 단일 운영 DB

3 명이 같은 DB 를 기준으로 개발하고, SSH · psql 로 운영 스키마를 직접 검증

4

작게 시작해 필요 시 확장

소규모 인스턴스로 비용을 관리하고, 부하에 맞춰 사양을 조정할 수 있는 구조

본인 수행

EC2 Ubuntu 에 PostgreSQL + pgvector 설치 → SQLite 스키마 · 데이터 마이그레이션 → SSH · psql 로 운영 스키마 전수 대조

MyTaskPilot > 운영 DB 정합성 · 보안 검증

문서가 아닌 운영 DB 와 실제 권한 흐름을 기준으로 검증

Case 1 ERD - 실제 운영 DB 스키마 불일치 전수 검증

문제

제작설계서 제출 전 최종 점검에서 설계 컬럼과 백엔드 구현 사이 다수 불일치 정황 (agent_executions 설계 12 vs 구현 6 컬럼, briefings 테이블 미구현)

원인

백엔드 담당이 스키마 · 인증 흐름을 변경할 때 DB 설계 담당에게 즉시 공유되지 않는 구조. 코드 리뷰만으로는 판단 불가한 항목 존재

조치

AWS EC2 운영 서버에 SSH 접속해 psql 로 6 개 테이블을 하나씩 대조. 구현 기준으로 ERD · 테이블 정의서 전면 수정, "아키텍처 변경은 즉시 문서화 · 공유" 원칙 합의

효과

6 / 6

6 개 테이블의 문서 · 운영 스키마 일치, 이후 구조 변경 시 즉시 문서화 · 공유 기준을 팀에 적용

Case 2 IDOR (권한 우회) 취약점 발견

문제

코드베이스 검토 중 클라이언트가 전달한 user_id 를 서버가 그대로 신뢰해 처리하는 구조 발견

원인

서버 측 사용자 식별이 없어 타 사용자 데이터 접근이 가능한 Insecure Direct Object Reference

조치

팀에 즉시 공유 → 서버가 JWT 로 사용자를 직접 식별하고, 권한 없는 요청은 403 을 반환하도록 전 엔드포인트에 방어 로직 적용

효과

전 엔드포인트

서버 측 권한 검증 체계 확보, refresh_token AES-256-GCM 암호화에도 기여

기능 12 개 항목 100% 구현과 실서비스 배포, 설계 문서와 운영 DB 6 개 테이블 완전 일치

기능 100% 구현

PostgreSQL 데이터 모델

pgvector 임베딩 차원

합성 검증 데이터 · 페르소나 3종

유사도 검색 (테스트 환경)

운영 DB 대조 완료 테이블

MyTaskPilot > 프로젝트 수행 및 산출물

데이터 모델과 RAG 검증을 중심으로, 설계 문서와 운영 DB의 일치까지 책임

01 분석

본인 주도 2 · 협업 1 / 전체 3

주도

요구사항 정의서

일정 · 할 일 · 이메일 이력 저장 범위

주도

시나리오 정의서

페르소나 3 중 기반 사용 시나리오

협업

기능 요구사항 정의

12 개 기능 · HITL 승인 원칙

02 설계

본인 주도 3 · 협업 1 / 전체 4

주도

ERD · 테이블정의서

users 중심 6 개 테이블 관계 설계

주도

pgvector 스키마 설계

vector(1536) · 코사인 유사도

주도

RAG 개인화 설계

text-embedding-3-small · top-3 반영

협업

서비스 구성도 · 메뉴 구성도

입력 → Agent → HITL → 저장 흐름

03 개발

본인 주도 3 · 협업 1 / 전체 5

주도

SQLite → PostgreSQL 전환

EC2 Ubuntu 설치 · 스키마 마이그레이션

주도

합성 데이터 · RAG 검증

226 건으로 개인화 동작 검증

주도

운영 DB 정합성 전수 검증

SSH 접속 · psql 스키마 대조 · 문서 정정

협업

OCR/STT · 보안 검증

정확도 검증 · IDOR 발견 / 공유

팀원

Agent · API · UI 구현

Function Calling · React 19

정량 성과 (최종 제출 2026.09.07 기준)

12 / 12

기능 100% 구현

6 테이블

PostgreSQL · pgvector 1536 차원

226 건

합성 검증 데이터 · 페르소나 3 중

0.1~0.2 초

pgvector 유사도 검색 (테스트 환경)

03

영화 예매 플랫폼

신용 리스크 예측 모델링 (알고리즘분석) · 영화 예매 플랫폼 (ERD 설계) · 식단 관리 웹 애플리케이션 (데이터 연동)

영화 예매 플랫폼 > 표준 기반 데이터 모델링

데이터모델링 교육 직후, 요구사항 정의부터 논리 · 물리 ERD, DB 구축, 예약 기능 구현까지 단독 수행

구분	본인 역할	기술 스택	서비스
3 학년 겨울방학 개인 프로젝트 · 2026.01~02 (2 개월)	단독 수행 — 요구사항 · 모델링 · DB 구축 · 구현	DA# Modeler · PostgreSQL · Node.js / Express · HTML / JS	movie-booking-platform- ui.vercel.app

상황 배운 모델링을 실제 서비스에 적용할 기회

- 데이터모델링 교육을 막 수료한 시점
교육에서 배운 절차를 실제 서비스 설계에 그대로 적용해 보기로 결정
- 영화 · 상영 · 좌석 · 예약이 얹힌 관계형 도메인
관계 설계가 조회 쿼리 복잡도와 예약 정합성을 좌우
- 화면 개발 경험 부족
DB · 백엔드 중심으로 역량을 쌓아 와 UI 구현은 미숙

해결 과제 설계 → 구현을 한 사이클로

- 1 표준 기반 모델링
단어 · 도메인 표준 정의 → 논리 ERD → 물리 ERD
- 2 무결성 · 조회 성능 확보
FK 제약조건과 조회 조건 기준 인덱스 전략 수립
- 3 설계를 기능으로 연결
JOIN 기반 예약 · 좌석 쿼리 → API → 화면까지 구현

액션 ① 모델링 → 구현 절차

요구사항 정의

표준 용어 · 도메인

논리 ERD

물리 ERD

PostgreSQL DDL

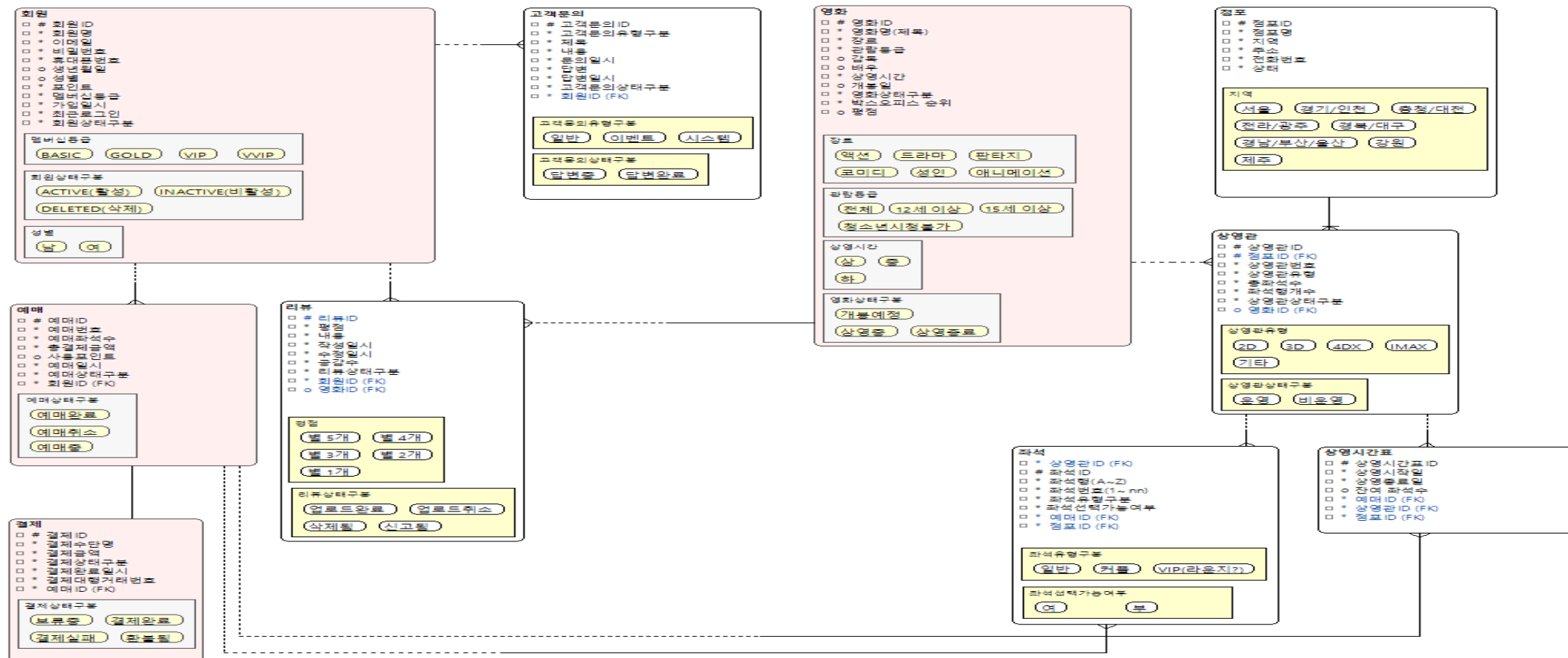
Express API

예약 · 좌석 쿼리

웹 화면

DA# Modeler 로 논리 · 물리 ERD 설계 → PostgreSQL 테이블 · 외래키 · 인덱스 구축 → JOIN 기반 예약 · 좌석 쿼리를 Express API 로 연결, 화면은 AI 프롬프트 기반 바이트코딩으로 구현

영화 예매 사이트 구현을 위한 회원, 영화, 상영관, 예매, 결제 ERD 설계



영화 예매 플랫폼 > 이슈 해결

모델링 · 쿼리 · 화면 구현 과정에서 부딪힌 문제와 해결

Case 1

용어 · 명명 규칙 불일치

문제 같은 의미의 속성이 테이블마다 다른 이름 · 타입으로 정의될 위험

원인 설계 초기에 명명 규칙과 도메인 기준이 없었음

조치 단어 · 도메인 표준을 먼저 정의하고 전 테이블 컬럼명 · 데이터 타입을 표준에 맞춰 통일

효과

표준 적용

일관된 명명 체계로 ERD · 쿼리 가독성 확보

Case 2

스키마로 인한 쿼리 복잡도

문제 예약 · 좌석 조회 시 여러 테이블 JOIN 이 얹혀 쿼리가 복잡해짐

원인 상영 · 좌석 · 예약 간 관계 설계가 실제 기능 흐름을 충분히 반영하지 못함

조치 기능 흐름 기준으로 관계 · 외래키를 재정의하고 조회 조건 컬럼에 인덱스 적용

효과

ERD 우선

이후 팀 프로젝트에서 ERD 를 먼저 확정하는 방식으로 이어짐

Case 3

화면 개발 경험 부족

문제 프론트엔드 경험이 적어 화면 구현에 일정이 지연될 위험

원인 DB · 백엔드 중심으로 역량을 쌓아 와 UI 구현 경험이 부족

조치 AI 프롬프트 기반 바이브코딩으로 화면을 구현하고, 본인은 DB · API 설계에 집중

효과

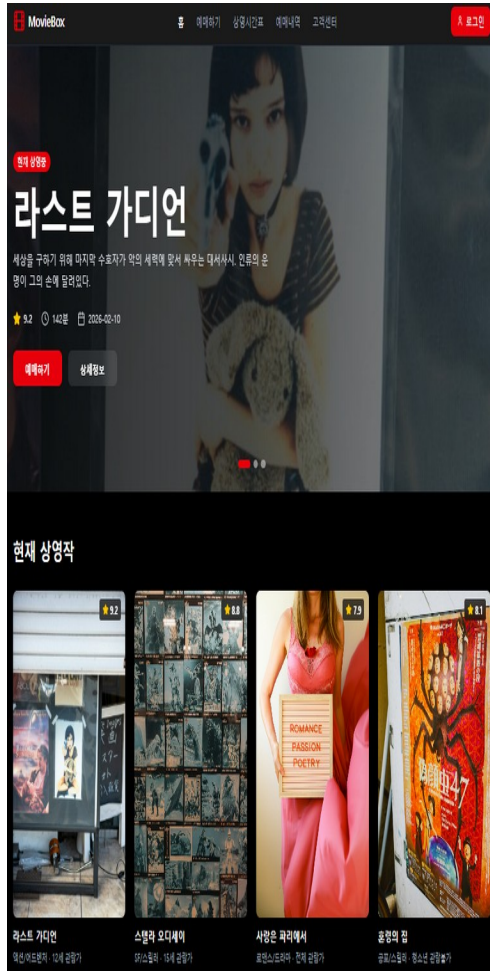
단독 완성

기획 · 화면까지 수행 영역 확장, 개발 생산성 향상

영화 예매 플랫폼 > 결과 · 화면

논리·물리 ERD → PostgreSQL 구축 → JOIN 기반 예약·좌석 기능을 화면까지 단독 구현

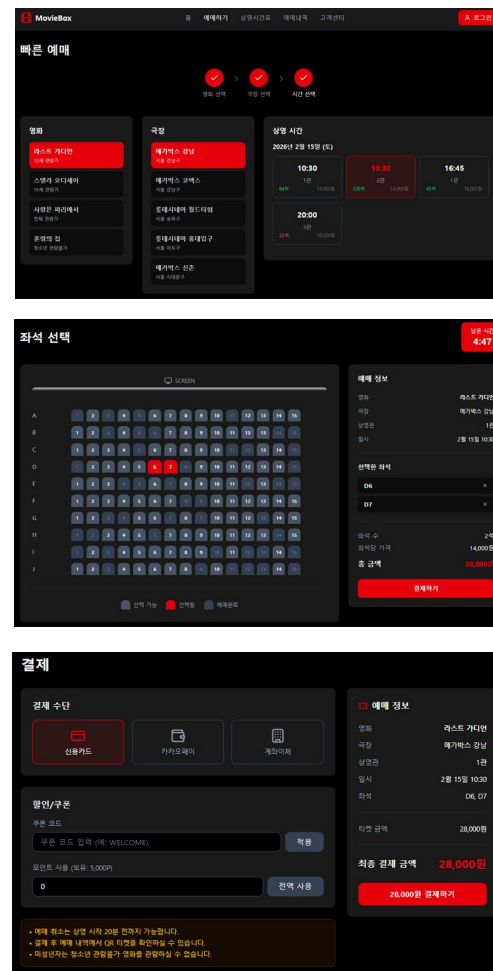
홈



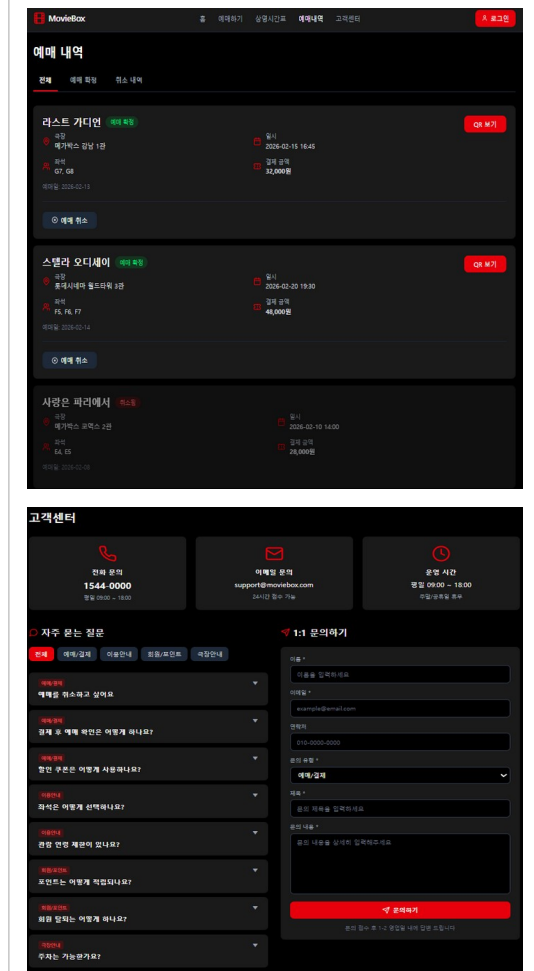
상영시간표



예매



예매내역 / 고객센터



영화 예매 플랫폼 > 프로젝트 수행 및 산출물

교육에서 배운 모델링 절차를 요구사항 → 논리 → 물리 → 구현 순서로 그대로 적용

01 분석

본인 주도 2 · 협업 0 / 전체 2

주도

요구사항 정의서

조회 · 예매 · 예약 확인 기능 정의

주도

업무 흐름 파악

영화 → 상영 → 좌석 → 예약 흐름

02 설계

본인 주도 4 · 협업 0 / 전체 4

주도

단어 / 도메인 표준 정의서

용어 · 데이터 타입 기준 수립

주도

논리 ERD

엔터티 · 관계 · 식별자 정의

주도

테이블정의서

테이블 · 컬럼 · 키 · 제약조건

주도

기능정의서, 화면설계서

예약 · 좌석 조회 조건 기준

03 개발

본인 주도 4 · 협업 0 / 전체 4

주도

DDL · 제약조건 구현

PostgreSQL 테이블 · 외래키

주도

Express API 구현

Node.js 백엔드 · DB 연동

주도

예약 · 좌석 쿼리

JOIN 기반 조회 · 예약 처리

주도

웹 화면 구현

HTML / JS · 바이브코딩 활용

이 프로젝트에서 얻은 것

표준화 습관

용어 · 도메인 표준을 먼저 정의

ERD 우선 설계

스키마가 구현 복잡도를 결정함을 체감

설계 → 구현 연결

모델이 쿼리 · API 로 이어지는 흐름 이해

AI 활용 생산성

비전문 영역은 바이브코딩으로 보완

Thank You

전예원 · Data Engineer Portfolio

DATA PIPELINE · POSTGRESQL · PGVECTOR · CHROMADB · AWS EC2 · ORACLE